

MusicGO

Plataforma de Streaming de Audio — Informe de la Fase 2

Cátedra: Programación Orientada a Objetos

Universidad: UCAB — Facultad de Ingeniería · Escuela de Ing. Informática

Profesor: Marcel J. Castro G.

Equipo: Equipo MusicGO

Semestre: 2026-25

Tema: Migración a GUI JavaFX con arquitectura MVC, SOLID y KISS

1. Introducción

Tras el éxito del motor de lógica de negocio de la Fase 1 (aplicación de consola), la Fase 2 lleva MusicGO a un **entorno gráfico moderno, seguro y escalable**. El sistema se refactorizó en profundidad bajo el patrón arquitectónico **MVC**, se dividió el control en dos perfiles (Administrador y Usuario final) y se incorporaron nuevas pasarelas de interacción: pagos multimodales, compartición de playlists, control parental y un panel dinámico de estadísticas.

Objetivo cumplido: migrar de consola a JavaFX manteniendo el dominio del negocio, separando estrictamente las capas y aplicando los principios SOLID, KISS, DRY y modularidad (máximo 300 líneas por archivo).

2. De la Fase 1 a la Fase 2

Aspecto	Fase 1	Fase 2
Interfaz	Consola de texto	GUI JavaFX (tema Apple Music light)
Arquitectura	Capas + servicios	MVC estricto en capas
Perfiles	Uno genérico	Administrador y Usuario final
Pagos	Compra directa	Pasarela multimodal (tarjeta / transferencia / saldo)
Restricciones	—	Control parental sobre contenido explícito
Colaboración	—	Compartir playlists entre usuarios
Estadísticas	Texto por usuario	Dashboard + Top de más reproducidos
Persistencia	Guardado al salir	Inmediata, en hilos de fondo
Build	<code>javac</code> manual	Maven + wrapper
Principios	DRY, KISS	+ SOLID completo

3. Arquitectura — MVC en capas

El sistema separa las responsabilidades en tres capas independientes. Las vistas no contienen lógica de negocio ni acceso a datos; los controladores coordinan; el modelo y los servicios encapsulan las reglas.

```
musicgo/
├─ MusicGoApp.java      Punto de entrada JavaFX (Application)
├─ app/                AppContext (composition root), Router, Vista
├─ modelo/ (+ pago/)   MODELO: entidades de dominio (sin JavaFX)
├─ interfaces/ (+ pago/) Contratos; interfaces de pago SEGREGADAS (ISP)
├─ excepciones/        Excepciones de dominio personalizadas
├─ persistencia/       Repositorio + Mapeadores JSON + persistencia async
├─ servicios/ (+ pago/) CONTROL/negocio: "gestores" + ProcesadorPago
├─ controlador/        CONTROLADORES MVC (vista <-> servicios)
└─ vista/ (ui/admin/usuario) VISTAS JavaFX (solo interfaz)
```

Capa MVC	Paquetes	Responsabilidad
Modelo	modelo, interfaces, excepciones, servicios, persistencia	Entidades, reglas de negocio y datos. Cero dependencias de JavaFX.
Vista	vista	Construye la interfaz, expone controles y dispara callbacks. Sin lógica.
Controlador	controlador, app	Escucha eventos, invoca servicios y traduce excepciones a alertas.

4. Principios de diseño aplicados

SOLID

Principio	Dónde se evidencia
SRP — Responsabilidad Única	Un gestor por tarea (<code>GestorCatalogo</code> , <code>GestorPlaylists</code> , <code>GestorAutenticacion</code> ...). Los <code>Mapeador*</code> separan el mapeo JSON de la lógica.
OCP — Abierto/Cerrado	Agregar un medio de pago o un tipo de audio/producto no obliga a modificar <code>ProcesadorPago</code> ni el reproductor: basta una clase nueva.
LSP — Sustitución de Liskov	<code>FondosInsuficientesException</code> hereda de <code>PagoRechazadoException</code> ; cualquier <code>MedioPago</code> es intercambiable.
ISP — Segregación de Interfaces	Pagos con interfaces pequeñas: <code>RequiereTarjeta</code> , <code>RequiereCuentaBancaria</code> , <code>UsaSaldoVirtual</code> . Ningún medio implementa métodos que no usa.
DIP — Inversión de Dependencias	Los controladores dependen de abstracciones; <code>AppContext</code> (composition root) cablea las implementaciones concretas.

KISS, DRY y modularidad

- **KISS:** clases pequeñas y enfocadas; el archivo más grande tiene 271 líneas.
- **DRY:** la fábrica `Componentes` evita repetir UI; los `Mapeador*` centralizan cada formato JSON; `Alertas` unifica los diálogos.
- **Modularidad:** 72 archivos, ninguno supera 300 líneas; sin monolitos.

5. Funcionalidades de la Fase 2

5.1 Doble perfil y autenticación

El arranque muestra una pantalla de login. Los administradores se validan contra `administradores.json` (no hay registro desde la interfaz: abstracción del registro). Los usuarios se registran con alias y correo, con **validación de unicidad en tiempo real**, e inician sesión por alias.

5.2 Back-office (módulo de administración)

Panel visual para dar de alta, listar, editar y eliminar canciones, podcasts y productos. Cualquier cambio se persiste y se **notifica a la app del usuario en tiempo real** mediante un patrón observador que no acopla el modelo a JavaFX.

5.3 Pasarela de pagos multimodal (ISP)

Al comprar un producto, el usuario elige entre **tarjeta de crédito**, **transferencia bancaria** o **saldo virtual MusicGo**. Cada medio aplica un flujo de validación distinto y solo implementa la interfaz que le corresponde. `ProcesadorPago` (patrón Strategy) es el único punto de cobro.

5.4 Control parental

Los audios tienen una clasificación (`APT0_TODO_PUBLICO` / `EXPLICIT0`). Si el perfil tiene el control parental activo y se intenta reproducir contenido explícito, `GestorReproduccion` lanza `ContenidoRestringidoException` y el controlador muestra una advertencia, bloqueando la reproducción sin tumbar la app.

5.5 Compartir playlist

Un usuario busca a otro por su alias y le comparte una playlist; el destinatario la ve en su sección "Compartidas conmigo". Se persiste de inmediato.

5.6 Dashboard dinámico

Tarjetas con tiempo de escucha, reproducciones, biblioteca y compras, más el **Top de contenidos más reproducidos** de la plataforma (contador global persistido en `reproducciones.json`). Incluye recarga de saldo virtual.

5.7 Persistencia en caliente con hilos

`ServicioPersistencia` guarda cada cambio en un **hilo daemon**, sin congelar la interfaz, y cierra de forma limpia al salir. Todo en JSON, con un parser/writer propio (sin librerías externas).

6. Mejoras concretas respecto a la Fase 1

- ✓ Interfaz gráfica completa que reemplaza el menú de consola.
- ✓ Separación MVC estricta: el modelo ya no imprime en consola (`reproducir()` → `creditos()` ; `mostrarDetalle()` → `detalle()`).
- ✓ Dos perfiles de acceso con interfaces independientes.
- ✓ Sistema de pagos extensible con interfaces segregadas (antes: compra directa).
- ✓ Robustez: excepciones personalizadas capturadas y convertidas en alertas gráficas.
- ✓ Persistencia inmediata y no bloqueante (antes: solo al salir).
- ✓ Build reproducible con Maven + wrapper (antes: `javac` a mano).
- ✓ Nuevas reglas de negocio: control parental, compartir, ranking global.

7. Verificación y métricas

Indicador	Resultado
Compilación (mvn clean compile)	BUILD SUCCESS
Prueba de lógica end-to-end (headless)	13 / 13 PASS
Archivos Java	72
Archivos que superan 300 líneas	0 (máximo: 271)
Dependencias externas de JSON	0 (parser/writer propios)

La prueba automatizada valida: carga de datos, login admin/usuario, bloqueo de contenido explícito, reproducción, pago con saldo, fondos insuficientes, compartir playlist, creación de playlists y CRUD del administrador.

8. Cómo ejecutar

```
./mvnw javafx:run      # macOS / Linux  
mvnw.cmd javafx:run    # Windows
```

Credenciales: admin / admin123 ; usuarios rocker99 , podcastlover (este con control parental).

9. Conclusión

MusicGO Fase 2 cumple todos los requerimientos del enunciado: GUI JavaFX, arquitectura MVC, doble perfil, pagos con ISP, control parental, compartición, dashboard y persistencia en hilos, manteniendo un código modular, optimizado y fiel a SOLID y KISS. La base queda preparada para crecer (nuevos medios de pago, tipos de contenido o pantallas) con cambios mínimos y localizados.